

Università di Pisa / Informatica Umanistica

Linguistica Computazionale II

Report



Studente: Jacopo Grandi

Matricola: 703340

Anno accademico: 2025/2026

3/06/2026

Indice

1	Introduzione	2
2	Costruzione del dataset	2
3	Primo modello: Classificatore basato su SVM lineari e n-grammi	2
3.1	Ulteriori sperimentazioni	4
3.1.1	5-grams di caratteri	4
3.1.2	Lunghezza testi per classe	5
3.1.3	Baseline con sole features di lunghezza	5
4	Secondo modello: Classificatore basato su Support Vector Machine (SVM) lineari informazioni linguistiche non lessicali.	6
4.1	Profiling-UD	7
4.2	Performance.	7
5	Terzo modello: Classificatore basato su SVM lineari e word embedding	8
5.1	Performance	9
6	Quarto modello: Fine-tuning di un modello di linguaggio neurale (Neural Language Model).	10
6.1	Performance	10
7	Conclusioni	11

1 Introduzione

Il progetto affrontato consiste nello sviluppo di un sistema di classificazione testuale per distinguere un testo scritto da umano da uno sintetico, ossia generato artificialmente da un large language model. Il progetto si divide in quattro punti differenti: la sperimentazione di quattro modelli di machine learning, sviluppati per assolvere allo stesso task: machine-generated text detection. I quattro modelli sono: 1) Classificatore basato su SVM lineari e n-grammi. 2) Classificatore basato su Support Vector Machine (SVM) lineari e informazioni linguistiche non lessicali. 3) Classificatore basato su SVM lineari e word embedding. 4) Fine-tuning di un modello di linguaggio neurale (Neural Language Model).

Il dataset su cui svolgere questo compito è quello del task Desegma-IT, proposto come shared task a EVALITA 2026

2 Costruzione del dataset

Dato il dataset originale del task Desegma-IT, che si presenta già separato in Train e Test Set, la consegna richiede di estrarre un sample di almeno 2000 documenti per il training set, 1000 documenti presi dal training set da cui creare un validation set e 1000 documenti estratti dal test set da utilizzare, appunto, come test set.

Questa procedura viene affrontata nel notebook “Corpus.ipynb”: per creare il dataset finale richiesto è stata svolta una breve parte preliminare di EDA, che ha rilevato le seguenti caratteristiche: il dataset originale Desegma è composto da circa 33mila documenti, perfettamente bilanciati tra umano [0] /macchina [1]: 16569 documenti esatti per ogni classe. Il primo punto della creazione del dataset è stato dunque inizializzare due variabili: una per la classe [0] (umani) “train_class0” e una per la classe [1] (macchine) “train_class1”. All’interno di queste due variabili è stato eseguito un sampling di rispettivamente 1000 e 1000 documenti, per un totale bilanciato di 2000 documenti di testi umani e di macchine per formare il training set. I due dataframe sono stati successivamente uniti in “df_train”.

A questo punto per il validation set, la logica utilizzata è stata la stessa, ossia, inizializzare due variabili una per la classe [0] e una per la classe [1], escludendo le porzioni già utilizzate per creare il training, così da evitare che ci sia sovrapposizione nel validation set. Per il test set si è seguita la stessa procedura del training, un sample di 500 documenti per classe, per un totale di 1000.

3 Primo modello: Classificatore basato su SVM lineari e n-grammi

Il primo modello è una linear support vector machine che prende come rappresentazione del testo diverse configurazioni di n-grammi. La configurazione migliore viene scelta testandola su una cross validation a 5 fold. Le features sono estratte dal testo tramite Stanza [3]

¹inizializzato per l'italiano, e rappresentate tramite una classe `Document` che ha al suo interno testo originale, token annotati e label. Le diverse configurazioni scelte sono: unigrammi, bigrammi e trigrammi di parola, unigrammi, bigrammi e trigrammi di lemma, unigrammi, bigrammi e trigrammi di pos, trigrammi, 4-grams e 5-grams di carattere. La cross validation a 5 fold sul training set ha restituito i seguenti risultati

Configurazione	F1	Standard deviation
('word', 1)	0.978	0.007462704962475109
('word', 2)	0.973	0.008349569580764965
('word', 3)	0.897	0.011370280318139308
('lemma', 1)	0.969	0.00584879981016869
('lemma', 2)	0.974	0.005515103752987804
('lemma', 3)	0.939	0.005687826437090686
('pos', 1)	0.923	0.011085336442628836
('pos', 2)	0.948	0.01067414167468555
('pos', 3)	0.950	0.006185790175668455
('char', 3)	0.968	0.008337124243243883
('char', 4)	0.971	0.004157077698008979
('char', 5)	0.978	0.008163615503023254

Tabella 1: Risultati della cross validation a 5 fold sul training set.

Unigrammi di parola e 5-grams di carattere sono parimerito le due configurazioni che performano meglio con un f1 di 0.978, per aggiungere informazione, è stata calcolata anche la standard deviation, che risulta essere più bassa sugli unigrammi di parola, che vengono quindi scelti come configurazione finale da testare.

Validation				
	precision	recall	f1-score	support
0	0.98	0.97	0.97	500
1	0.97	0.98	0.97	500
accuracy			0.97	1000
macro avg	0.97	0.97	0.97	1000
weighted avg	0.97	0.97	0.97	1000

Tabella 2: Risultati su validation set.

¹Stanza è un toolkit NLP open-source sviluppato dallo Stanford NLP Group. Permette di processare testi in molte lingue attraverso una pipeline neurale che include tokenizzazione, lemmatizzazione, POS tagging, analisi morfologica e dependency parsing. Nel progetto è stato usato per annotare linguisticamente i testi italiani prima dell'estrazione degli n-grammi.

Test				
	precision	recall	f1-score	support
0	0.82	0.98	0.89	500
1	0.97	0.78	0.87	500
accuracy			0.88	1000
macro avg	0.89	0.88	0.88	1000
weighted avg	0.89	0.88	0.88	1000

Tabella 3: Risultati su test set.

Il modello mostra nel classification report della validation di essere in grado di predire con un'accuratezza del 97% entrambe le classi, c'è invece un calo sostanziale nel test set, dove l'accuracy generale cala intorno all'88%, ma andando più a fondo possiamo notare come sia la classe [0] ossia quella dei testi umani a subire il calo più notevole passando dal 98% di precision sul validation set all'82% sul test. Segnale che il modello fa difficoltà a generalizzare sui testi scritti da umani che potrebbe essere causata da un leggero overfitting. Non risulta evidente ancora sul validation set, che essendo estratto dal training set probabilmente si presenta molto simile al training set.

3.1 Ulteriori sperimentazioni

Dato la natura del task, è stato deciso di testare anche i trigrammi di pos, nonostante non abbiano avuto la performance migliore sulla cross validation, è lecito pensare che possano essere comunque una configurazione capace di catturare molta informazione sullo stile di scrittura. Nonostante le performance rimangano pressoché identiche agli unigrammi di parola sulla validation, il calo su test set per la classe [0] è ancora più notevole, scendendo addirittura a un 77% di precision.

Test				
	precision	recall	f1-score	support
0	0.77	0.94	0.85	500
1	0.93	0.73	0.81	500
accuracy			0.83	1000
macro avg	0.85	0.83	0.83	1000
weighted avg	0.85	0.83	0.83	1000

Tabella 4: Risultati su test set con trigrammi di POS.

3.1.1 5-grams di caratteri

Sono stati testati anche i 5-grams di carattere dato il loro score, parimerito con gli unigrammi di parola sulla cross validation. Anche qui però il drop sul test set per la classe [0] scende fino al 77% di precision.

Test				
	precision	recall	f1-score	support
0	0.77	0.98	0.86	500
1	0.97	0.71	0.82	500
accuracy			0.84	1000
macro avg	0.87	0.84	0.84	1000
weighted avg	0.87	0.84	0.84	1000

Tabella 5: Risultati su test set con 5-grams di caratteri.

3.1.2 Lunghezza testi per classe

È stata verificata la lunghezza dei testi per classe, nel tentativo di capire se la performance degli unigrammi di parola, dato che sono stati usati conteggi grezzi, potesse in qualche modo essere influenzata dalla struttura del dataset.

split	label	n_tokens			n_chars			n_words_space		
		mean	median	std	mean	median	std	mean	median	std
train	0	628.13	539.0	354.22	3206.30	2787.5	1824.96	503.29	433.0	281.79
train	1	312.14	273.5	180.21	1643.06	1427.0	961.39	253.22	219.0	146.71
validation	0	588.06	493.0	324.68	3013.08	2519.5	1671.27	471.75	396.0	259.28
validation	1	320.05	277.0	180.36	1689.78	1465.0	984.23	259.56	223.0	147.75
test	0	692.93	507.5	1420.62	3546.24	2595.0	7566.44	559.01	408.0	1219.44
test	1	233.39	244.0	63.63	1245.96	1297.0	326.32	190.93	198.5	50.05

Tabella 6: Lunghezza dei testi per classe nei tre split.

Anche il Mann-Whitney conferma che la differenza è statisticamente molto significativa in tutti gli split:

Split	Media cl. 0	Media cl. 1	Mediana cl. 0	Mediana cl. 1	Mann-Whitney p-value
TRAIN	628.13	312.14	539.0	273.5	1.51×10^{-211}
VALIDATION	588.06	320.05	493.0	277.0	1.21×10^{-97}
TEST	692.93	233.39	507.5	244.0	1.65×10^{-129}

Tabella 7: Risultati del test di Mann-Whitney sulla lunghezza dei testi.

gli unigrammi non rappresentano soltanto il vocabolario dei testi, ma incorporano indirettamente anche la lunghezza, perché i documenti più lunghi generano più conteggi.

3.1.3 Baseline con sole features di lunghezza

Per rafforzare questa ipotesi è stata creata una baseline che utilizza solo features di lunghezza estratte dalle features annotate in precedenza: numero di token = len(words), numero di caratteri = len(text) e numero di spazi tra carattere = len(text.split). La

baseline addestrata con queste features performa già notevolmente bene, segno che la performance degli unigrammi di parola beneficia notevolmente di una peculiarità del dataset in questione, risultando più sensibile alla variazione di lunghezza dei documenti.

Split	Classe	precision	recall	f1-score	support
Validation	0	0.86	0.64	0.73	500
Validation	1	0.71	0.89	0.79	500
Validation	accuracy			0.77	1000
Validation	macro avg	0.79	0.77	0.76	1000
Validation	weighted avg	0.79	0.77	0.76	1000
Test	0	1.00	0.61	0.76	500
Test	1	0.72	1.00	0.84	500
Test	accuracy			0.81	1000
Test	macro avg	0.86	0.80	0.80	1000
Test	weighted avg	0.86	0.81	0.80	1000

Tabella 8: Baseline con sole feature di lunghezza.

Come ultima prova è stata testata una frase generata sinteticamente da Chat-gpt. Nel prompt è stato richiesto di generare una frase che sembri umana. Il testo generato è il seguente: "Non so bene perché, ma negli ultimi mesi ho iniziato a fare caso a dettagli che prima mi sfuggivano completamente. Per esempio, quando torno a casa la sera, mi accorgo subito se qualcuno ha spostato una sedia o se la luce del corridoio è rimasta accesa più del solito. Non è una cosa importante, anzi probabilmente non significa niente, però mi dà la sensazione che la giornata abbia lasciato una traccia anche prima che io riesca davvero a pensarci. Forse è solo stanchezza, oppure il fatto che passo troppe ore davanti al computer e poi cerco nel mondo reale qualcosa che non abbia bisogno di essere aggiornato, salvato o ricaricato. In ogni caso, mi sembra strano quanto certe abitudini piccole finiscano per raccontare meglio di noi rispetto alle cose che diciamo apertamente." Il modello, addestrato con unigrammi di parola, classifica il testo come umano nonostante sia un testo breve, più corto della mediana dei testi artificiali e molto distante da quella dei testi umani. Per la sola lunghezza ci si aspetterebbe una maggiore tendenza verso la classe macchina: il fatto che lo classifichi come umano mostra che gli unigrammi di parola non sono un puro rilevatore di lunghezza, ma combinano lunghezza e segnale lessicale.

4 Secondo modello: Classificatore basato su Support Vector Machine (SVM) lineare informazioni linguistiche non lessicali.

Il secondo classificatore è una support vector machine lineare che prende come rappresentazione del testo informazioni linguistiche non lessicali, ossia, il testo è rappresentato solamente tramite le features estratte tramite profiling-UD.

4.1 Profiling-UD

Profiling-UD [1] è uno strumento per il linguistic profiling, cioè per descrivere un testo non tanto in base al suo contenuto, ma in base alla sua forma linguistica. L'idea di fondo è trasformare un testo in una rappresentazione fatta di feature linguistiche: per esempio lunghezza del documento, lunghezza media delle frasi, distribuzione delle categorie grammaticali, densità lessicale, struttura verbale, relazioni sintattiche e uso della subordinazione. Lo strumento lavora in due fasi. Prima il testo viene annotato linguisticamente: vengono svolte operazioni come segmentazione in frasi, tokenizzazione, POS tagging, lemmatizzazione e dependency parsing. Poi, a partire da questa annotazione, Profiling-UD estrae una serie di misure quantitative che descrivono diversi aspetti della struttura linguistica del testo. le feature vengono estratte a partire dal framework Universal Dependencies, uno schema di annotazione che rappresenta in modo standard categorie grammaticali e relazioni sintattiche. Questo permette di descrivere il testo attraverso proprietà linguistiche astratte, come la distribuzione dei POS, le relazioni di dipendenza o la struttura della subordinazione, invece di basarsi direttamente sulle parole presenti nel testo.

4.2 Performance.

La support vector machine con profiling ud è il modello che, dopo la rete neurale BERT, performa meglio, questo perché per la natura del task, il profiling-UD è una rappresentazione ideale, in quanto estremamente rappresentativa dell'aspetto stilistico del testo, andando a individuare con un'accuracy dell'88% su test set quando il documento è umano o sintetico.

Validation				
	precision	recall	f1-score	support
0	0.94	0.94	0.94	500
1	0.94	0.94	0.94	500
accuracy			0.94	1000
macro avg	0.94	0.94	0.94	1000
weighted avg	0.94	0.94	0.94	1000

Tabella 9: SVM lineare con feature Profiling-UD: risultati su validation set.

Test				
	precision	recall	f1-score	support
0	0.84	0.93	0.88	500
1	0.92	0.82	0.87	500
accuracy			0.88	1000
macro avg	0.88	0.88	0.88	1000
weighted avg	0.88	0.88	0.88	1000

Tabella 10: SVM lineare con feature Profiling-UD: risultati su test set.

Le feature più rilevanti mostrano una distinzione abbastanza netta tra le due classi. I testi umani risultano associati soprattutto a lunghezza maggiore, maggiore densità lessicale e maggiore varietà formale. Inoltre, alcune misure legate alla lunghezza media delle catene preposizionali e subordinate spingono verso la classe umana, suggerendo una maggiore articolazione strutturale. I testi generati, invece, sembrano essere caratterizzati da una maggiore presenza relativa di elementi funzionali e relazioni sintattiche ricorrenti, come punteggiatura, determinanti, adposizioni, modificatori avverbiali e soggetti nominali. Più che indicare una complessità sintattica “profonda”, queste feature sembrano descrivere una maggiore regolarità grammaticale della scrittura generata.

Feature	Coefficiente	Classe associata
n_tokens	-2.4214	Classe 0 – umano
dep_dist_advmod	1.8710	Classe 1 – generato
upos_dist_ADV	-1.5215	Classe 0 – umano
lexical_density	-1.4564	Classe 0 – umano
n_prepositional_chains	1.3119	Classe 1 – generato
ttr_form_chunks_200	-1.2974	Classe 0 – umano
dep_dist_det	-1.2333	Classe 0 – umano
upos_dist_NUM	-1.1199	Classe 0 – umano
upos_dist_PUNCT	1.0165	Classe 1 – generato
upos_dist_PRON	-1.0001	Classe 0 – umano
upos_dist_DET	0.9312	Classe 1 – generato
dep_dist_flat:foreign	-0.8928	Classe 0 – umano
dep_dist_case	-0.7994	Classe 0 – umano
upos_dist_VERB	-0.7751	Classe 0 – umano
avg_prepositional_chain_len	-0.7623	Classe 0 – umano
avg_subordinate_chain_len	-0.7147	Classe 0 – umano
upos_dist_ADP	0.6047	Classe 1 – generato
upos_dist_PROPN	-0.5881	Classe 0 – umano
dep_dist_acl:relcl	0.5429	Classe 1 – generato
dep_dist_nsubj	0.5012	Classe 1 – generato

Tabella 11: Venti feature più rilevanti del classificatore SVM con Profiling-UD. I coefficienti negativi spingono verso la classe 0, mentre quelli positivi verso la classe 1.

5 Terzo modello: Classificatore basato su SVM lineari e word embedding

Il terzo classificatore è una Support vector machine lineare che utilizza una rappresentazione vettoriale del testo costruita a partire dagli embeddings italiani itWaC messi a disposizione

da Italian NLP Lab² [2].

Sono state testate tre diverse rappresentazioni del testo: la prima configurazione è la media di tutti gli embedding: quindi con embedding a 128 dimensioni, come in questo caso, abbiamo un vettore di 128 dimensioni che prova a rappresentare il significato medio del documento. La seconda configurazione consiste nella media di soli aggettivi, nomi e verbi, costruendo una rappresentazione più incentrata sul contenuto semantico. La terza configurazione calcola la media separata di nomi, verbi e aggettivi e poi le aggrega in un unico vettore di dimensioni $128 * 3$, risultando la migliore f1 macro sulla validation.

Strategia di aggregazione	F1 macro val.
Media di tutti gli embeddings	0.926
Media solo di ADJ, NOUN, VERB	0.903
Media separata per ADJ/NOUN/VERB + concatenazione	0.929

Tabella 12: Confronto tra le strategie di aggregazione degli embeddings sul validation set.

5.1 Performance

La migliore configurazione, ossia, media separata di nomi, verbi, aggettivi e poi concatenati, è stata testata sul test, restituendo i seguenti risultati:

Test set				
	precision	recall	f1-score	support
0	0.73	0.94	0.82	500
1	0.92	0.65	0.76	500
accuracy			0.80	1000
macro avg	0.83	0.80	0.79	1000
weighted avg	0.83	0.80	0.79	1000

Tabella 13: Risultati su test set della terza configurazione basata su media separata degli embeddings e concatenazione.

Tra i modelli testati, quello basato su embeddings aggregati risulta il meno performante sul test set. Questo risultato è coerente con la natura della rappresentazione: la media degli embeddings tende a catturare soprattutto il contenuto semantico generale del documento, ma perde molte informazioni rilevanti per un task di riconoscimento dello stile.

²Gli itWaC embeddings sono rappresentazioni vettoriali di parole italiane addestrate con word2vec sul corpus itWaC, un corpus di circa un miliardo di parole costruito a partire dal Web italiano. Gli embeddings hanno dimensionalità 128 e sono stati ottenuti senza analisi linguistica profonda del corpus: il preprocessing indicato dagli autori prevede principalmente tokenizzazione e normalizzazione delle forme.

6 Quarto modello: Fine-tuning di un modello di linguaggio neurale (Neural Language Model).

Il quarto classificatore è *BERT base italian cased*, modello transformer già addestrato su grandi quantità di testo italiano, più una testa per la classificazione.

I dati sono stati gestiti tramite una classe figlia di Dataset di PyTorch, adattata al dataframe del progetto. Per ogni documento, la classe applica il tokenizer del modello, impone padding e troncamento fino a 512 token e restituisce `input_ids`, `attention_mask` e label. I DataLoader permettono poi di organizzare gli esempi in mini-batch durante training, validation e test.

Questi i report del modello sulle tre epoche:

Epoca	Train loss	Val loss	Accuracy val.	Macro F1 val.
1	0.2077	0.0400	0.99	0.99
2	0.0493	0.0338	0.99	0.99
3	0.0176	0.0178	0.99	0.99

Tabella 14: Andamento di BERT sul validation set durante le tre epoche.

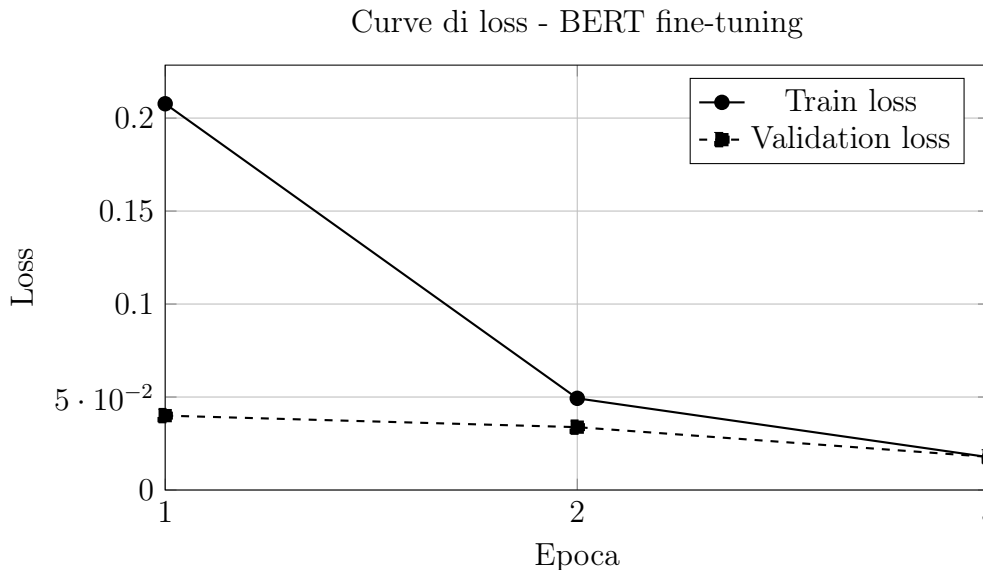


Figura 1: Andamento della train loss e della validation loss durante le tre epoche.

Al termine della terza epoca l'errore continua a scendere sia su Train che su Validation, segno che il modello sta imparando a generalizzare bene.

6.1 Performance

La performance di BERT alla terza epoca sul test set è la seguente:

Test set – Epoca 3				
	precision	recall	f1-score	support
0	0.94	0.97	0.95	500
1	0.97	0.94	0.95	500
accuracy			0.95	1000
macro avg	0.95	0.95	0.95	1000
weighted avg	0.95	0.95	0.95	1000

Tabella 15: BERT sul test set alla terza epoca.

7 Conclusioni

Nel complesso, il modello migliore risulta essere BERT, che raggiunge la macro F1 più alta sul test set. Il risultato è coerente con la natura del modello: a differenza delle rappresentazioni precedenti, BERT non lavora su feature aggregate o conteggi locali, ma costruisce rappresentazioni contestuali dei token, riuscendo quindi a catturare meglio informazioni lessicali, sequenziali e stilistiche.

Tra i modelli lineari, il classificatore basato su Profiling-UD risulta particolarmente interessante, perché ottiene performance elevate pur usando informazioni non lessicali. Questo conferma che, per un task di distinzione tra testi umani e testi generati, la forma linguistica del testo è molto informativa: lunghezza, distribuzione dei POS, relazioni sintattiche, densità lessicale e strutture subordinate contribuiscono a descrivere differenze di stile tra le due classi.

La SVM con unigrammi di parola ottiene un risultato comparabile a Profiling-UD sul test set, ma la sua interpretazione è più delicata. L’analisi sulla lunghezza dei documenti mostra infatti che nel dataset i testi umani sono sistematicamente più lunghi dei testi generati. Poiché gli unigrammi sono rappresentati tramite conteggi grezzi, questa configurazione non cattura soltanto il vocabolario dei testi, ma anche un segnale indiretto legato alla lunghezza. Per questo motivo le sue buone prestazioni sembrano dipendere sia da informazione lessicale sia da una peculiarità specifica del dataset.

Il modello basato su embeddings aggregati è invece quello meno efficace. La media degli embeddings permette di rappresentare il contenuto semantico generale del documento, ma tende a perdere informazioni importanti per il riconoscimento dello stile, come ordine delle parole, pattern sintattici, punteggiatura e uso delle parole funzionali. Questo lo rende meno adatto al task rispetto sia alle feature linguistiche esplicite sia al fine-tuning di BERT.

Modello	Macro F1 test
SVM + unigrammi di parola	0.88
SVM + Profiling-UD	0.88
SVM + embeddings aggregati	0.79
BERT fine-tuning	0.95

Tabella 16: Confronto finale tra i quattro modelli sul test set.

Riferimenti bibliografici

- [1] Dominique Brunato, Andrea Cimino, Felice Dell’Orletta, Simonetta Montemagni, and Giulia Venturi. Profiling-ud: A tool for linguistic profiling of texts. In *Proceedings of the 12th Conference on Language Resources and Evaluation (LREC 2020)*, pages 7145–7151, Marseille, France, 2020. European Language Resources Association.
- [2] Andrea Cimino, Lorenzo De Mattei, and Felice Dell’Orletta. Multi-task learning in deep neural networks at evalita 2018. In *Proceedings of the 6th Evaluation Campaign of Natural Language Processing and Speech Tools for Italian (EVALITA 2018)*, Turin, Italy, 2018. CEUR.org.
- [3] Peng Qi, Yuhao Zhang, Yuhui Zhang, Jason Bolton, and Christopher D. Manning. Stanza: A python natural language processing toolkit for many human languages. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pages 101–108. Association for Computational Linguistics, 2020.